

CO1

Question 1

Title

Application: Hierarchical Tree Query Engine

Aim

To solve the recurrence relation

$$T(n) = T\left(\frac{n}{2}\right) + \log n$$

using the **Substitution Method**, determine the recursion depth, derive the asymptotic complexity, and analyze the scalability of the algorithm.

Problem Statement

A tree-processing engine reduces the problem size by half at each recursive level and performs logarithmic work for query evaluation.

Apply the substitution method to solve the recurrence

$$T(n) = T(n/2) + \log n$$

Expand the relation step-by-step, explain the recursion depth, derive the final asymptotic complexity, analyze its scalability, and interpret the result using asymptotic notation.

Theory

The **Substitution Method** is used to solve recurrence relations by repeatedly expanding the recursive equation until the base case is reached. The expanded expression is then simplified to obtain the time complexity of the algorithm.

Since the problem size is reduced by half in every recursive call, the recursion depth is logarithmic.

Step-by-Step Solution

Given Recurrence

$$T(n) = T(n/2) + \log n$$

First Expansion

$$T(n) = T(n/4) + \log(n/2) + \log n$$

Second Expansion

$$T(n) = T(n/8) + \log(n/4) + \log(n/2) + \log n$$

Third Expansion

$$T(n) = T(n/16) + \log(n/8) + \log(n/4) + \log(n/2) + \log n$$

General Form

After **k** recursive calls,

$$T(n) = T(n/2^k) + \sum_{i=0}^{k-1} \log\left(\frac{n}{2^i}\right)$$

Recursion Depth

The recursion stops when

$$\frac{n}{2^k} = 1$$

Therefore,

$$2^k = n$$

Taking logarithm,

$$k = \log n$$

Hence, the recursion depth is

$$\boxed{\log n}$$

Simplifying the Summation

Each term can be written as

$$\log(n/2^i) = \log n - i$$

Therefore,

$$T(n) = \sum_{i=0}^{\log n - 1} (\log n - i)$$

This becomes

$$(\log n) + (\log n - 1) + (\log n - 2) + \cdots + 1$$

The sum of the first m integers is

$$\frac{m(m+1)}{2}$$

where

$$m = \log n$$

Hence,

$$T(n) = \frac{\log n (\log n + 1)}{2}$$

Ignoring constants and lower-order terms,

$$T(n) = \Theta((\log n)^2)$$

Algorithm

1. Start with the recurrence relation.
2. Expand the recurrence recursively.
3. Determine the recursion depth.
4. Express the recurrence as a summation.
5. Simplify the summation.
6. Derive the asymptotic complexity.

CODE: -

```
#include <stdio.h>

#include <math.h>

// Recursive function for T(n) = T(n/2) + log2(n)
double T(int n)
{
    if (n <= 1)
        return 1;
    return T(n / 2) + log2(n);
}

int main()
{
    int n;

    printf("Enter value of n: ");
    scanf("%d", &n);
```

```
double result = T(n);  
printf("\nT(%d) = %.2lf\n", n, result);  
printf("Asymptotic Complexity = Theta((log n)^2)\n");  
return 0;
```

Output

Enter value of n: 16

T(16) = 11.00

Asymptotic Complexity = Theta((log n)^2)

=== Code Execution Successful ===

Analysis

- The input size is reduced by half during every recursive call.
- The recursion depth is $\Theta(\log n)$.
- Each recursive level performs $\Theta(\log n)$ work.
- Therefore, the total running time is

$$\Theta((\log n)^2)$$

This complexity is highly efficient and scales well for large hierarchical tree structures because the running time grows much slower than linear or quadratic algorithms.

Time Complexity

$$\Theta((\log n)^2)$$

Space Complexity

The recursion stack contains $\Theta(\log n)$ recursive calls.

Therefore,

$$\boxed{\Theta(\log n)}$$

Applications

- Hierarchical Tree Query Processing
- Binary Search Trees
- Segment Trees
- File System Navigation
- Decision Trees
- Network Routing Algorithms

Result

The recurrence relation

$$T(n) = T(n/2) + \log n$$

was successfully solved using the **Substitution Method**. The recursion depth is $\Theta(\log n)$, and the final asymptotic time complexity is

$$\boxed{\Theta((\log n)^2)}$$

The algorithm is highly scalable and efficient for processing large hierarchical tree structures.